

Module 3.1: Hadoop Architecture and HDFS

Contents

Data Formats	1
Analysing Data with Hadoop	2
Scaling Out	3
Hadoop Streaming	4
Hadoop Pipes	4
Hadoop Distributed File System (HDFS)	5
HDFS Java Interface.....	6
HDFS Data Flow	7

Data Formats

Data format refers to the way data is organized, stored, and represented in a computer system. The main types of data formats are:

- 1. Structured Data Formats:** Structured data is organized in a fixed format with predefined rows and columns.
 - Follows a predefined schema.
 - Stored in tables with rows and columns.
 - Easy to search and analyze.
 - Commonly used in relational databases.

Examples

- SQL Database Tables
- Excel Spreadsheets
- CSV Files

- 2. Semi-Structured Data Formats:** Semi-structured data does not follow a strict table structure but contains tags or metadata for organization.
 - Flexible schema.
 - Self-describing data.
 - Easier to modify than structured data.
 - Suitable for hierarchical information.

Examples

- XML
- JSON
- HTML

- 3. Unstructured Data Formats:** Unstructured data has no predefined format or organization.
 - Does not follow a fixed schema.
 - Difficult to process using traditional databases.
 - Large volume of data is generated daily.
 - Requires specialized tools for analysis.

Examples

- Images
- Audio Files
- Videos
- Emails
- Social Media Posts

Importance of Data Formats in Big Data

- Helps organize and store data efficiently.
- Improves data processing speed.
- Enables easy data integration from multiple sources.
- Supports analytics and decision-making.
- Reduces storage and management complexity.
- Helps select appropriate big data tools and technologies.

Analysing Data with Hadoop

Need for Data Analysis: Data analysis is the process of examining large amounts of data to discover useful information, patterns, and trends.

- Helps organizations make informed decisions.
- Identifies hidden patterns and relationships.
- Improves business performance and efficiency.
- Supports forecasting and prediction.
- Enables better customer understanding.
- Helps detect anomalies and fraud.

Hadoop for Data Processing

Hadoop is an open-source framework designed for storing and processing large datasets across clusters of computers.

Features of Hadoop for Data Processing

- Processes massive volumes of data efficiently.
- Uses distributed storage through HDFS.
- Employs MapReduce for parallel computation.
- Provides fault tolerance through replication.
- Supports scalability by adding more nodes.

Working of Hadoop Data Processing

1. Data is stored in HDFS.
2. Data is divided into blocks.
3. Blocks are distributed across DataNodes.
4. MapReduce processes the data in parallel.
5. Results are combined and returned to the user.

Batch Processing

Batch processing is a method of processing large volumes of data together as a single batch instead of processing data individually.

Characteristics

- Processes data in groups.
- Suitable for large datasets.
- Does not require immediate results.
- Efficient resource utilization.

- Bank statement generation
- Log file analysis

Advantages

- High throughput.
- Cost-effective.
- Suitable for repetitive tasks.

Examples

- Payroll processing

Parallel Processing

Parallel processing is the technique of executing multiple tasks simultaneously using multiple processors or machines.

- Divides work into smaller tasks.
- Executes tasks at the same time.
- Reduces processing time.
- Improves system performance.
- Data is distributed across multiple nodes.
- Each node processes its portion independently.
- Results are combined to produce the final output.

Hadoop and Parallel Processing

Advantages

- Faster execution.
- Better resource utilization.
- Supports large-scale data analysis.

Advantages of Hadoop in Data Analysis

- Handles very large datasets efficiently.
- Provides distributed storage and processing.
- Supports fault tolerance through data replication.
- Scales easily by adding more nodes.
- Processes data in parallel for faster results.
- Cost-effective because it uses commodity hardware.
- Supports structured, semi-structured, and unstructured data.
- Open-source and widely adopted in industry.

Scaling Out

Scaling Out is the process of increasing system capacity by adding more machines (nodes) to a distributed system instead of upgrading existing hardware. **Key Points**

- Also known as Horizontal Scaling.
- Commonly used in Hadoop clusters.
- Distributes workload among multiple nodes.
- Improves performance and storage capacity.

1) **Horizontal Scaling:** refers to adding additional servers or nodes to a system to handle increasing data and processing requirements. **Example:** If a Hadoop cluster with 5 nodes becomes overloaded, 5 more nodes can be added to increase processing power and storage.

- New nodes can be added easily.
- Workload is distributed across nodes.
- Provides better fault tolerance.
- Supports large-scale applications.

2) **Vertical Scaling:** refers to increasing the resources (CPU, RAM, storage, etc.) of an

existing server to handle increasing data and processing requirements. **Example:** If a server has 8 GB RAM and 4 CPU cores, it can be upgraded to 32 GB RAM and 16 CPU cores to improve performance.

- Hardware resources of a single machine are upgraded.
- Processing power and memory are increased.
- Easy to implement without adding new servers.
- Limited by the maximum capacity of the machine.

Scale Up (Vertical Scaling)	Scale Out (Horizontal Scaling)
Increases resources of an existing machine.	Adds more machines to the system.
Limited by hardware capacity.	Easily expandable by adding nodes.
More expensive hardware required.	Uses commodity hardware.
Single point of failure may exist.	Better fault tolerance.
Suitable for small systems.	Suitable for Big Data environments.

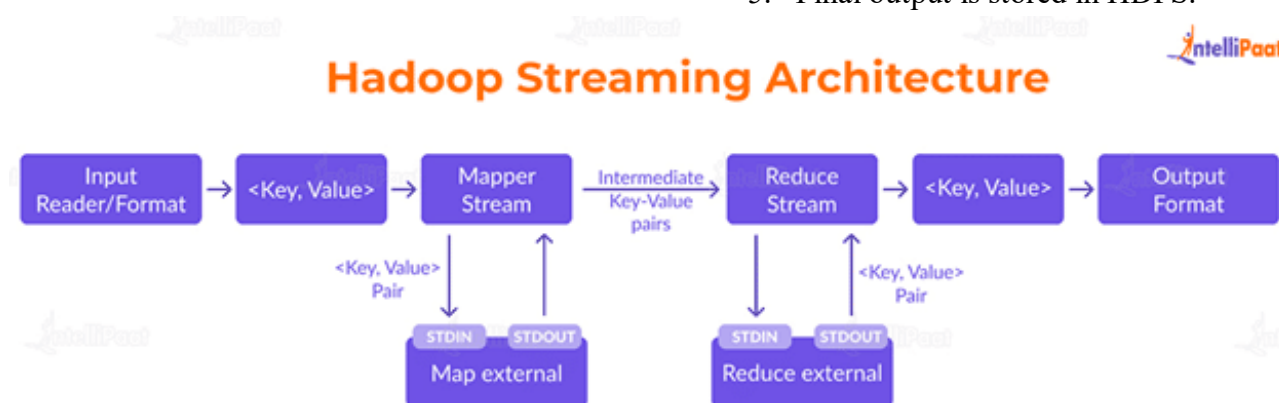
Benefits of Scaling Out

- Increases storage and processing capacity.
- Supports handling of massive datasets.
- Improves fault tolerance and reliability.
- Reduces system downtime.
- Cost-effective using commodity hardware.
- Enables parallel data processing.
- Provides flexible and easy expansion.

Hadoop Streaming

Hadoop Streaming is a utility that allows users to create MapReduce programs using any programming language that can read from standard input and write to standard output. **Examples of Languages:** Python, Ruby, Perl, Shell Scripts, PHP. **Steps**

1. Input data is provided to the Mapper.
2. Mapper processes the data using a chosen programming language.
3. Hadoop sorts and groups intermediate results.
4. Reducer processes grouped data.
5. Final output is stored in HDFS.



Features

- Supports multiple programming languages.
- Works with standard input and output.
- Easy integration with existing scripts.
- Compatible with Hadoop MapReduce.
- Suitable for rapid application development.

Advantages

- No need to write Java code.
- Supports language flexibility.

- Easy to learn and implement.
- Reuses existing programs and scripts.
- Faster development of MapReduce applications.

Applications

- Log file analysis.
- Text processing.
- Data transformation.
- Data filtering.
- Web data analysis.

Hadoop Pipes

Hadoop Pipes is a framework that allows developers to write Hadoop MapReduce applications in C++ instead of Java. **Working**

1. Input data is stored in HDFS.
2. Hadoop launches C++ Mapper programs.
3. Intermediate data is shuffled and sorted.

4. C++ Reducer programs process grouped data.
5. Output is stored in HDFS.

Features

- Supports MapReduce development in C++.
- Provides communication between Hadoop and C++ programs.
- Suitable for high-performance applications.
- Uses the Hadoop framework for execution.
- Integrates with HDFS and MapReduce.

Advantages

- Better performance for compute-intensive tasks.

- Allows reuse of existing C++ code.
- Suitable for developers familiar with C++.
- Provides efficient memory management.
- Supports large-scale distributed processing.

Applications

- Scientific computing.
- Data mining.
- Image processing.
- High-performance analytics.
- Large-scale computation tasks.

Hadoop Distributed File System (HDFS)

HDFS is a distributed file system designed to store very large files across multiple machines while providing high throughput and fault tolerance. **Design Components**

1) NameNode

- Master node of HDFS.
- Maintains metadata about files and directories.
- Keeps track of block locations.
- Handles client requests.
- Does not store actual file data.

2) DataNode

- Worker node of HDFS.
- Stores actual data blocks.
- Performs read and write operations.
- Sends heartbeat messages to the NameNode.
- Replicates blocks as instructed by the NameNode.

3) Blocks

- Files are divided into fixed-size blocks.

- Default block size is typically 128 MB or 256 MB.
- Blocks are distributed across DataNodes.
- Large files can be stored efficiently.

4) Replication

- Multiple copies of each block are stored.
- Default replication factor is 3.
- Ensures fault tolerance.
- Prevents data loss due to node failures.

5) Rack Awareness

- Hadoop places replicas across different racks.
- Improves fault tolerance.
- Reduces network congestion.
- Ensures data availability during rack failures.

Objectives of HDFS

- Store huge volumes of data efficiently.
- Provide fault tolerance through replication.

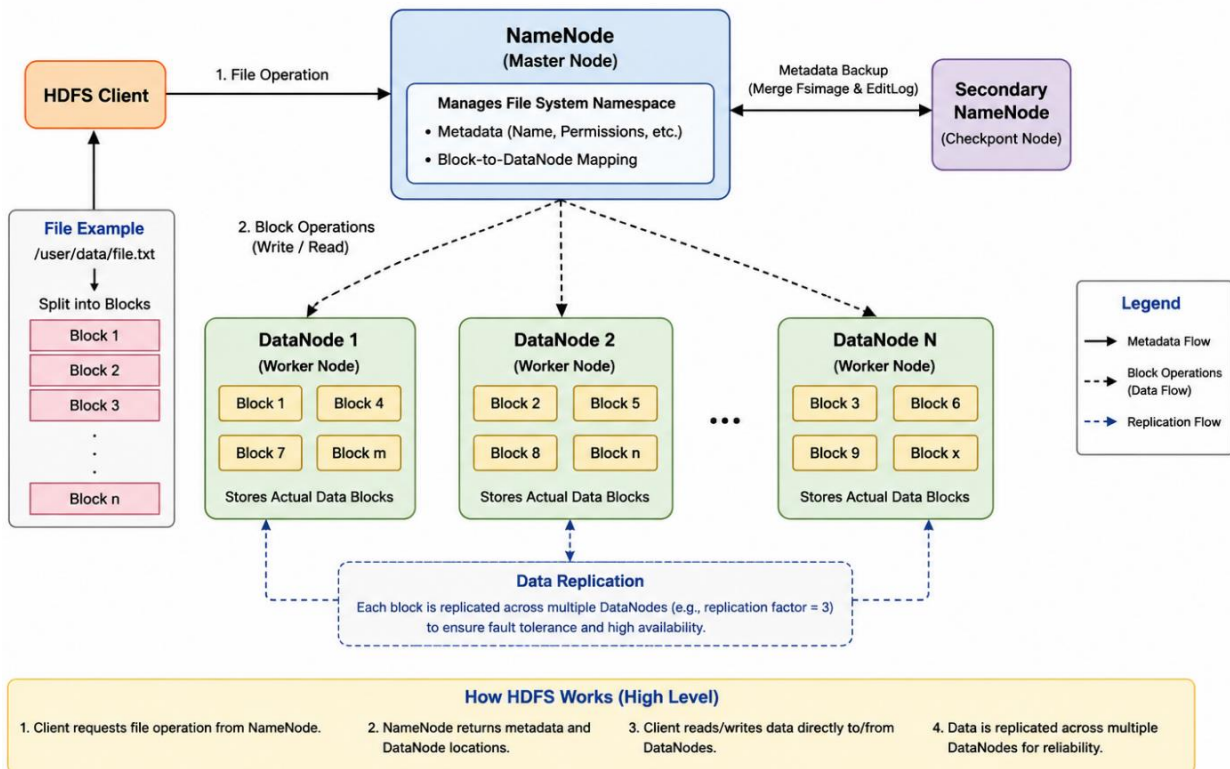
- Support distributed storage.
- Deliver high data throughput.
- Enable scalability by adding nodes.
- Ensure reliable data access.

- High availability and reliability.
- High-throughput data access.
- Automatic recovery from failures.
- Scalability through cluster expansion.
- Cost-effective storage using commodity hardware.

Key Features

- Distributed storage architecture.
- Large file support.
- Fault tolerance through replication.

HADOOP DISTRIBUTED FILE SYSTEM (HDFS)



HDFS Java Interface

The HDFS Java Interface provides APIs that allow Java applications to interact with HDFS for storing, retrieving, and managing files. **Uses**

- Access files in HDFS.
- Create and delete files.
- Read and write data.
- Manage directories.

File Operations

Common file operations supported by the HDFS Java API:

- Create files.
- Open files.
- Read files.
- Write files.
- Delete files.
- Rename files.
- Copy files.
- Create directories.

Example Operations

create()
open()

delete()
rename()

mkdirs()

Read/Write Operations

Read Operation

1. Client requests file from NameNode.
2. NameNode returns block locations.
3. Client reads blocks from DataNodes.
4. Data is returned to the application.

3. Client sends data to DataNodes.
4. Data is replicated.
5. Metadata is updated by NameNode.

Advantages

- Efficient data access.
- Supports large files.
- Automatic fault tolerance.
- Distributed storage management.

Write Operation

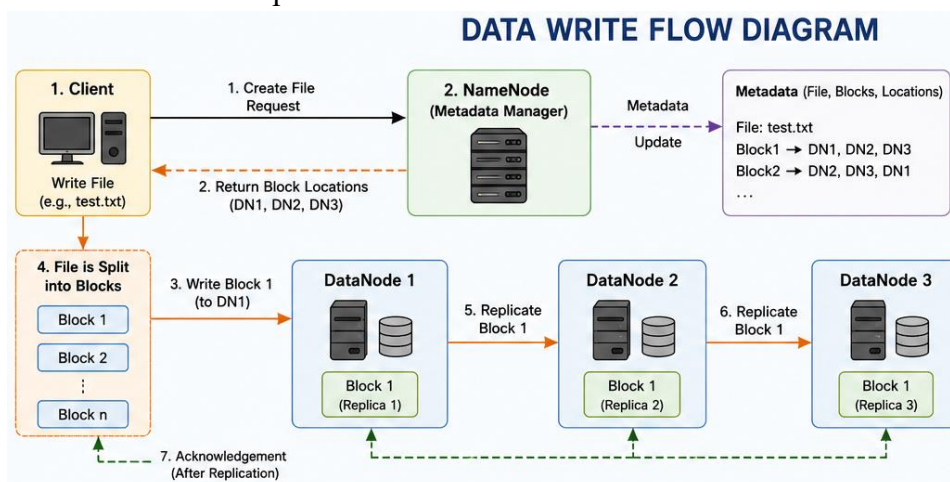
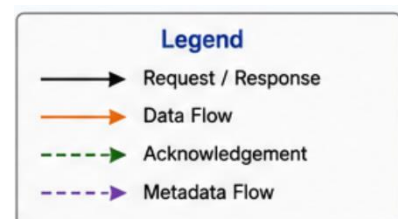
1. Client requests file creation.
2. NameNode allocates blocks.

HDFS Data Flow

Data Write Flow

When a client writes a file to HDFS, the following steps occur:

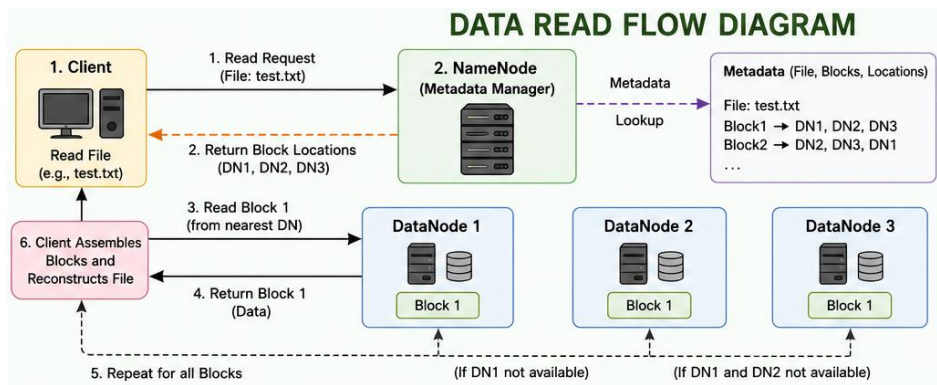
1. Client sends a file creation request to the NameNode.
2. NameNode verifies permissions and creates metadata.
3. NameNode provides DataNode locations.
4. File is split into blocks.
5. Client sends the first block to DataNode1.
6. DataNode1 forwards it to DataNode2.
7. DataNode2 forwards it to DataNode3.
8. Replication is completed.
9. Acknowledgments are returned to the client.
10. NameNode updates metadata.



Data Read Flow

When a client reads a file from HDFS, the following steps occur:

1. Client requests file information from the NameNode.
2. NameNode returns block locations.
3. Client connects to the nearest DataNode.
4. Data blocks are read.
5. Blocks are combined into the original file.
6. File is returned to the client.



Advantages of HDFS Data Flow

- High-speed data access.
- Fault-tolerant operation.
- Efficient block management.
- Supports large-scale distributed systems.
- Optimizes network utilization.